

1. Performance of waveform coding using PCM

MAT LAB CODE

```
clc;
clear all;
close all;
n=input('Enter for n-bit PCM system : ');
n1=input('Enter Sampling Frequency : ');
t=0:pi/n1:4*pi;
ActualSignal=sin(t)+3;
subplot(3,1,1);
plot(ActualSignal);
title('Analog Signal');
subplot(3,1,2);
stem(ActualSignal);
grid on;
title('Sampled Signal');
L=2^n;
Vmin=round(min(ActualSignal));
Vmax=round(max(ActualSignal));
StepSize=(Vmax-Vmin)/L;
QuantizationLevels=Vmin:StepSize:Vmax;
codebook=Vmin-(StepSize/2):StepSize:Vmax+(StepSize/2);
[ind,q]=quantiz(ActualSignal,QuantizationLevels,codebook);
NonZeroInd=find(ind~=0);
ind(NonZeroInd)=ind(NonZeroInd)-1;
BelowVminInd=find(q==Vmin-(StepSize/2));
q(BelowVminInd)=Vmin+(StepSize/2);
subplot(3,1,3);
stem(q);
grid on;
title('Quantized Signal');
figure;
TransmittedSig=de2bi(ind,'left-msb');
```

```

SerialCode=reshape(TransmittedSig.',1,size(TransmittedSig,1)
)*size(TransmittedSig,2));
subplot(2,1,1);
grid on;
stairs(SerialCode);
axis([0 40 -2 2]);
title('Transmitted Signal');
ReceivedCode=reshape(SerialCode,n,length(SerialCode)/n);
index=bi2de(ReceivedCode','left-msb');
q=(StepSize*index);
q=q+(Vmin+(StepSize/2));
subplot(2,1,2);
grid on;
plot(q);
title('Demodulated Signal');
for Nvar=1:10
Lvar=2^Nvar;
StepSizevar=(Vmax-Vmin)/Lvar;
QLevelvar=Vmin:StepSizevar:Vmax;
code=Vmin(StepSizevar/2):StepSizevar:Vmax+(StepSizevar/2
);
[ind,x]=quantiz(ActualSignal,QLevelvar,code);
a(Nvar)=snr(x,ActualSignal);
end
figure;
Nvar=1:10;
plot(Nvar,a,'-o');
grid on;
xlabel('Number of Bits');
ylabel('SNR (dB)');
title('SNR vs Number of Bits');

```

- Pulse shaping and match filtering

MATLAB CODE

```
clc;
clear all;
close all;
rolloff=0.4;
span=10;
sps=10;
M=4;
k=log2(M);
BP_data=2*randi([0 1],10,1)-1;
rectfilt=rcosdesign(rolloff,span,sps,'normal');
%rectfilt=rcosdesign(rolloff,span,sps,'sqrt');
fvtool(rectfilt,'impulse');
UP_s=upfirdn(BP_data,rectfilt,sps,1);
figure;
plot(UP_s);
EbNo=100;
snr=EbNo+10*log10(k)-10*log10(sps);
rxSignal=awgn(UP_s,snr);
figure;
plot(rxSignal);
rxSignal=rxSignal+randn(size(rxSignal))*0.01;
figure;
plot(rxSignal);
rxfilt=upfirdn(rxSignal,rectfilt,1,sps);
rxfilt=rxfilt(span+1:end-span);
figure;
stem(BP_data,'filled');
hold on;
plot(rxfilt,'r');
legend('Transmitted data','Received signal');
figure;
plot(rxSignal);
title('filtered signal through AWGN');
```

- Error performance of bpsk

Matlab code

```
clc;
clear all;
close all;
numBits=10^5;
bits=randi([0,1],numBits,1);
bpskSignal=2*bits-1;
EbNo_dB=0:10;
EbNo=10.^(EbNo_dB/10);
for i=1:length(EbNo_dB)
N0=1/EbNo(i);
noise=sqrt(N0/2)*randn(size(bpskSignal));
receivedSignal=bpskSignal+noise;
receivedBits=receivedSignal>0;
error(i)=sum(bits~=receivedBits);
end
ber=error/numBits;
theoryber=0.5*erfc(sqrt(EbNo));
semilogy(EbNo_dB,ber,'bo-');
hold on;
semilogy(EbNo_dB,theoryber,'ro-');
hold off;
grid on;
legend('Simulated BER','Theoretical BER');
title('BPSK Error Performance');
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate');
```

- Error performance of QPSK

Matlab code

```
clc;
clear all;
close all;
numBits=10^5;
bits=randi([0,1],numBits,1);
bitPairs=reshape(bits,2,[]).';
qpskSignal=(1-2*bitPairs(:,1))+1j*(1-2*bitPairs(:,2));
qpskSignal=qpskSignal/sqrt(2);
EbNo_dB=0:10;
EbNo=10.^(EbNo_dB/10);
for i=1:length(EbNo_dB)
N0=1/(2*EbNo(i));
noise=sqrt(N0/2)*(randn(size(qpskSignal))+1j*randn(size(qpskSignal)))
;
receivedSignal=qpskSignal+noise;
detectedBits=zeros(size(bitPairs));
detectedBits(:,1)=real(receivedSignal)<0;
detectedBits(:,2)=imag(receivedSignal)<0;
detectedBits=reshape(detectedBits.',[],1);
error(i)=sum(bits~=detectedBits);
end
ber=error/numBits;
theoryber=0.5*erfc(sqrt(EbNo));
semilogy(EbNo_dB,ber,'bo-');
hold on;
semilogy(EbNo_dB,theoryber,'ro-');
hold off;
grid on;
legend('Simulated BER','Theoretical BER');
title('QPSK Error Performance');
xlabel('Eb/No (dB)');
ylabel('Bit Error Rate');
```

- **FM Generation and Demodulation Using PLL**

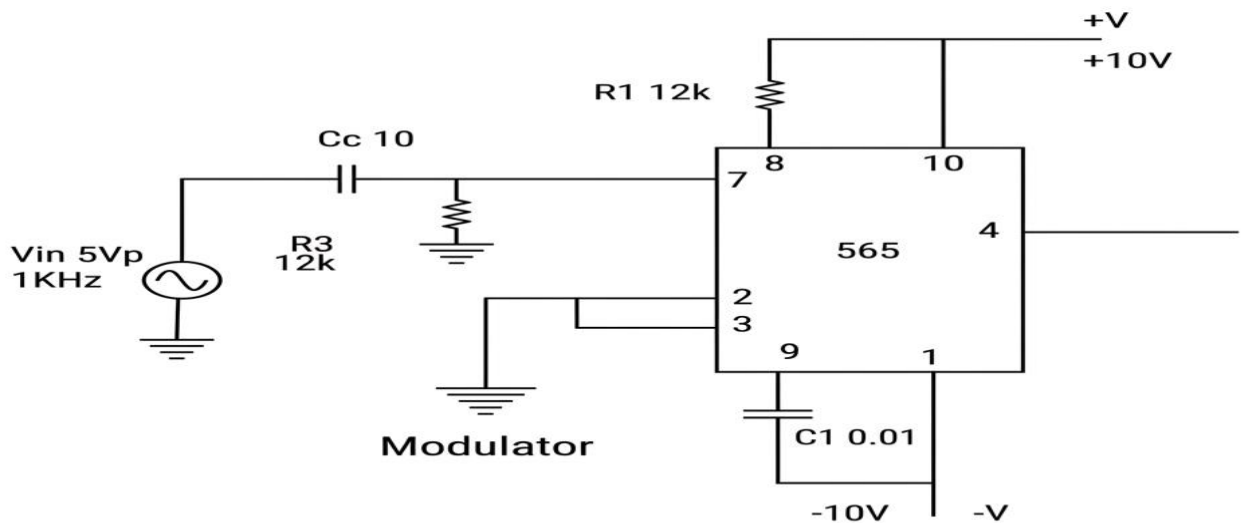
Aim:

Components: Breadboard, IC 565, resistors, capacitors, signal generator, power supply, and CRO.

Theory: In FM, the carrier frequency varies according to the modulating signal. PLL using IC 565 is used for FM generation and demodulation. The VCO generates FM signals, and the phase detector converts frequency variations into voltage variations to recover the original signal.

Diagram

1.FM modulator



DESIGN

Let $V+ = +10V$ and $V- = -10V$

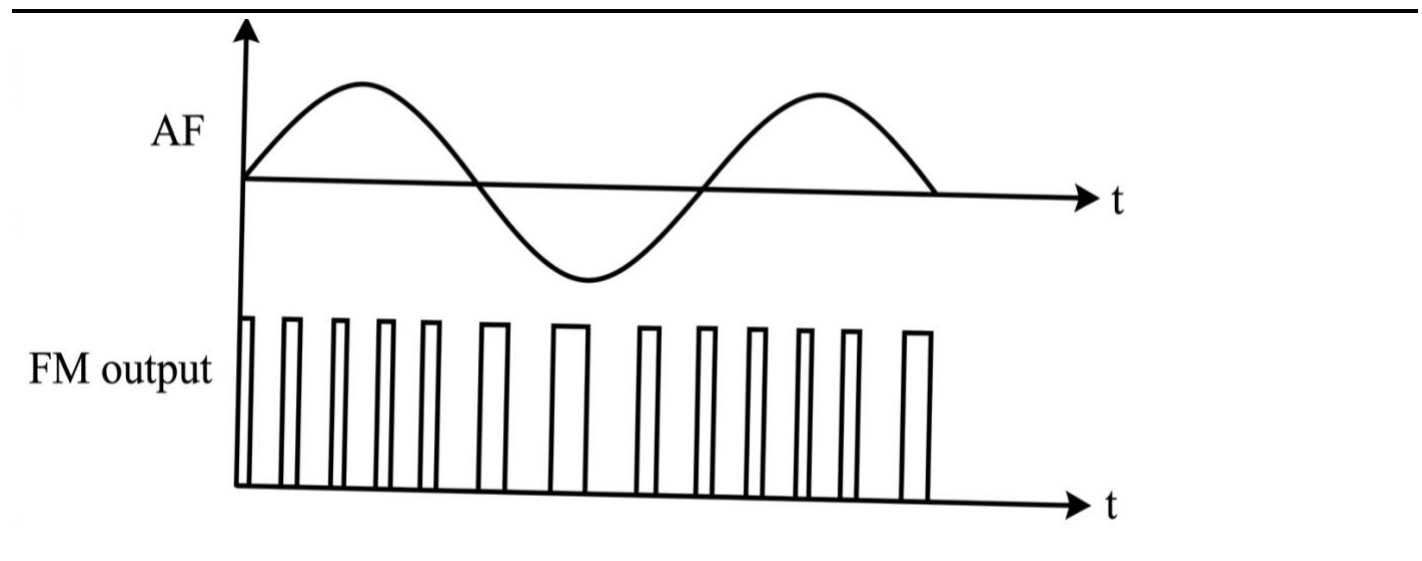
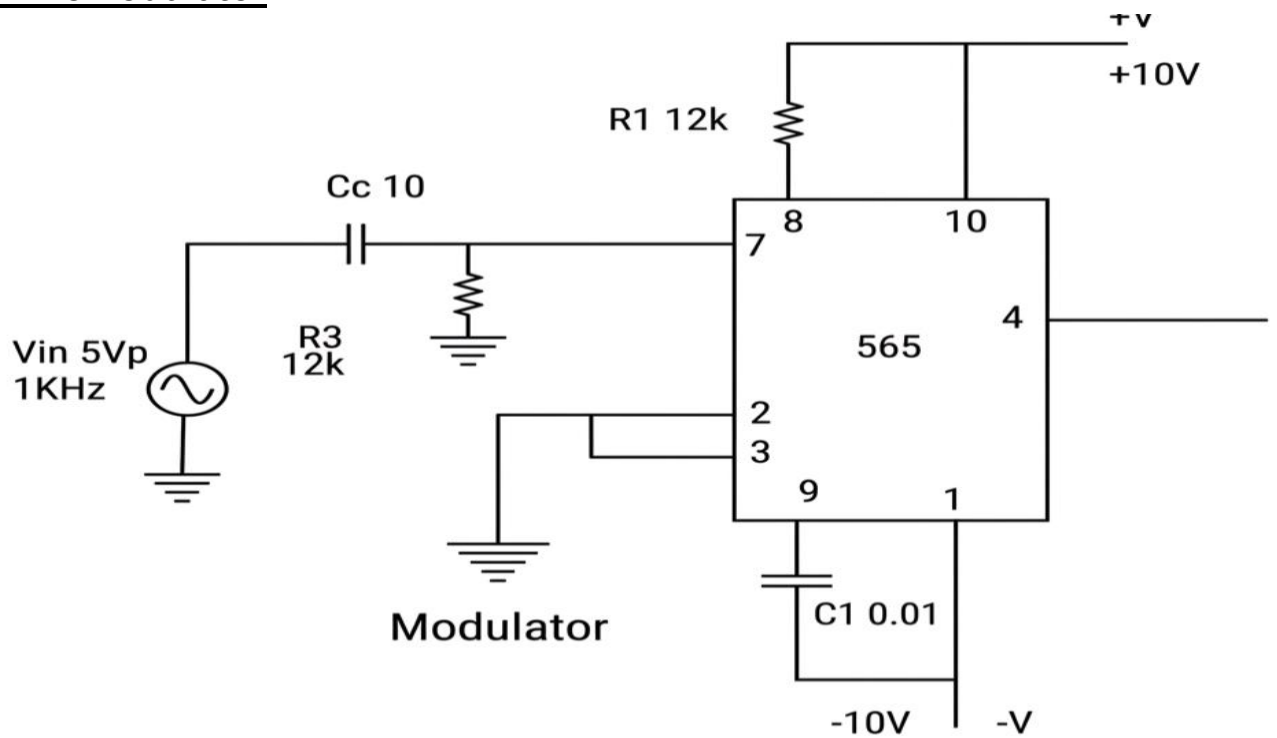
Let the centre frequency of the FM be $f_0 = \frac{1.2}{4R_1C_1} = 2.5kHz$

Take $C_1 = 0.01\mu F$. Then $R_1 = 12k$.

The value of R_1 satisfies the required condition $2k < R_2 < 20k$

Take $C_3 = 0.01\mu F$ and $C_2 = 10\mu F$.

2.FM Demodulator



- **GENERATION AND DETECTION OF DELTA MODULATED SIGNAL**

Aim:

Components

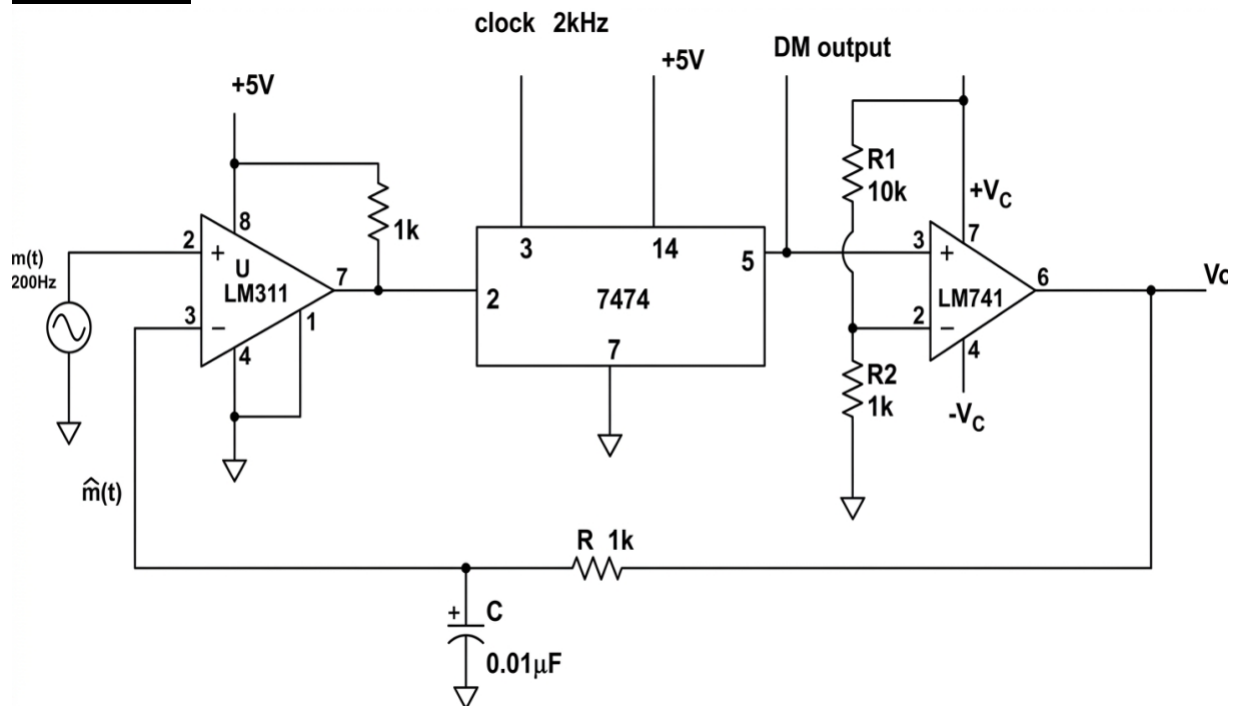
Op-amp, LM311, 7474 IC, resistors, capacitors, signal generator, breadboard, power supply, and CRO.

Theory

In Delta Modulation (DM), only the change in signal amplitude is encoded using one bit. Comparator compares the input and approximated signal. The D flip-flop samples the output, and an RC low pass filter recovers the analog signal.

CIRCUIT DIAGRAM

a) modulator



DESIGN

Let the input signal amplitude be 5 V and frequency be 200 Hz.
That is $m(t) = 5 \sin 400\pi t$

Maximum slope of $m(t) = 2\pi fA = 2\pi \times 200 \times 5 = 2000\pi$

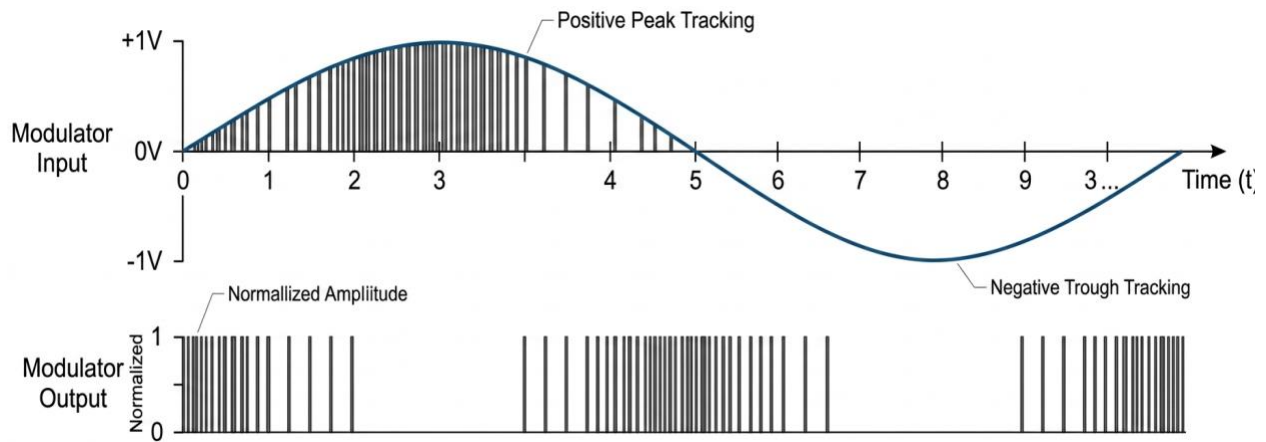
To avoid slope over load error,

$$V_{CC}/RC > E_m W_m$$

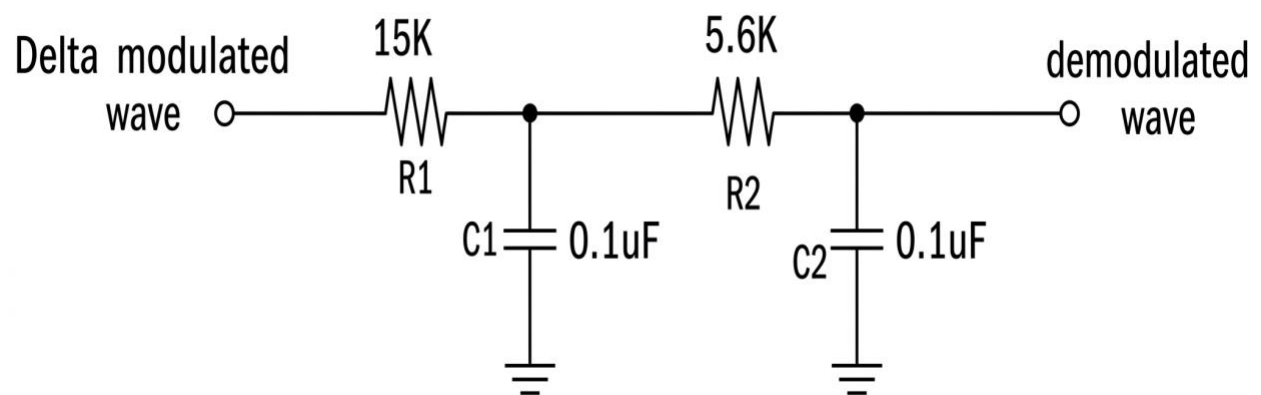
Selecting $V_{CC} = 15 \text{ V}$ and $C = 0.01 \mu\text{F}$. We get $R < 228 \text{ k}\Omega$.

Take $R = 100 \text{ k}\Omega$

Take $R_2 = 1 \text{ k}\Omega$, then $R_1 = 10 \text{ k}\Omega$



b)Demodulator

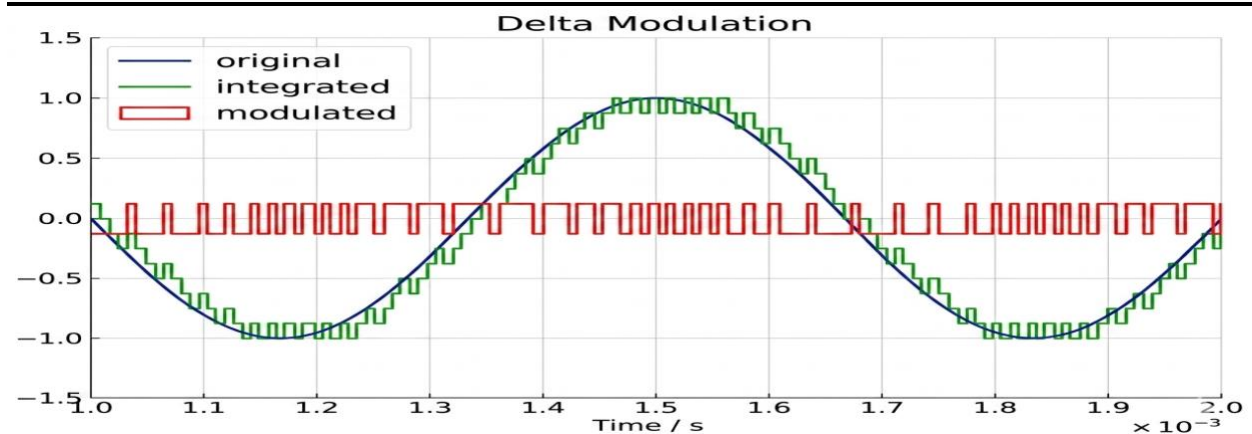


DESIGN

In demodulator, Low pass

$$\text{filter: } f_m < 1/2\pi R_2 C_2$$

Take $C_2 = 0.1\mu\text{F}$, then we get $R_2 = 5.6 \text{ k}$



- **Eye diagram**

Mat lab code

```
clc;
clear all;
close all;
rolloff_1=0.1;
rolloff_5=0.5;
span=10;
sps=8;
N=10;
BP_data=2*randi([0 1],N,1)-1;
rctfilt1=rcosdesign(rolloff_1,span,sps,'sqrt');
rctfilt5=rcosdesign(rolloff_5,span,sps,'sqrt');
UP_s1=upfirdn(BP_data,rctfilt1,sps,1);
UP_s5=upfirdn(BP_data,rctfilt5,sps,1);
figure;
subplot(2,1,1);
plot(UP_s1);
subplot(2,1,2);
plot(UP_s5);
figure;
eyediagram(UP_s1,2*sps);
title('alpha=0.1');
figure;
eyediagram(UP_s5,2*sps);
title('alpha=0.5');
```

Options

Title: FM Modulation

Author: Sultan Mohyuddin

Output Language: Python

Generate Options: QT GUI

Variable

ID: samp_rate

Value: 480k

Variable

ID: freq

Value: 104.8M

PlutoSDR Source

IIO context URI: 192.168.2.1

LO Frequency: 101.1M

Sample Rate: 480k

Buffer size: 16.384k

Quadrature: True

RF DC Correction: True

BB DC Correction: True

Gain Mode (RX1): Slow Attack

Filter Configuration: Auto

RF Bandwidth (Hz): 500k

Low Pass Filter

Decimation: 2

Gain: 1

Sample Rate: 480k

Cutoff Freq: 100k

Transition Width: 44.1k

Window: Hamming

FM Demod

Channel Rate: 240k

Audio Decimation: 5

Deviation: 75k

Audio Pass: 15k

Audio Stop: 20k

Gain: 1

Tau: 75u

Audio Sink

Sample Rate: 48 kHz

QT GUI Frequency Sink

FFT Size: 512

Center Frequency (Hz): ...freq

Bandwidth (Hz): 400k

QT GUI Range

ID: tun_freq

Label: UHD Freq (MHz)

Default Value: 104.8M

Start: 87.5M

Stop: 108M

Step: 100k

- FM RECEIVER

- FM TRANSMITTER

